

ICT1011

Computer Organisation and Architecture

Chapter 1

Fundamentals of Computer Architecture

Chapter Contents

- ✓ History and Evolution of Computers
- ✓ Computer Hardware Decomposition
- ✓ Classes of Computers
- ✓ Basic operations and requirements
- ✓ Components inside a Computer
- ✓ Example-Register to Register data transfer

Chapter Contents

✓ History and Evolution of Computers

✓ Computer Hardware Decomposition

✓ Classes of Computers

✓ Basic operations and requirements

✓ Components inside a Computer

✓ Example-Register to Register data transfer

History and Evolution of Computers

Early days of the computer

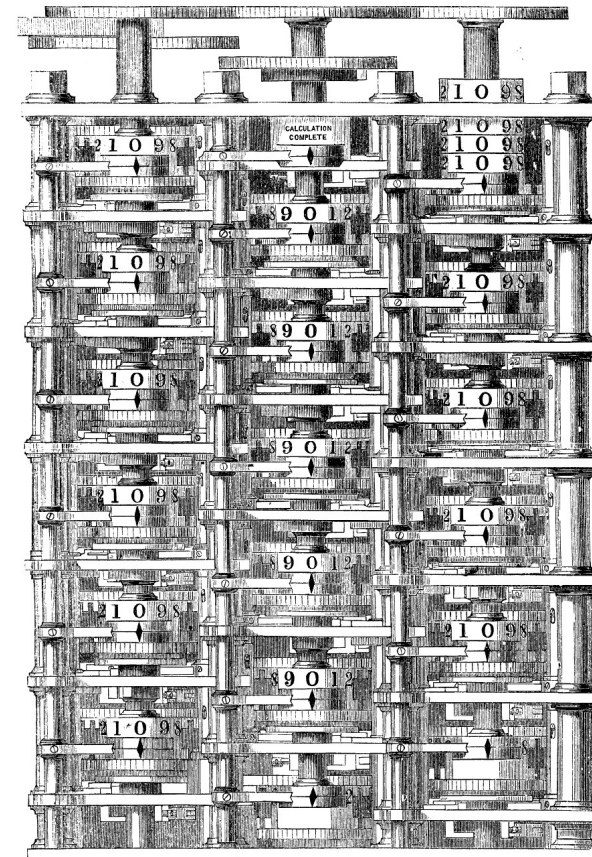
1834, Cambridge, UK

Charles Babbage built the *Difference Engine*, and then the *Analytical Difference Engine*.

This was a fully programmable digital computer, but was mechanical rather than electronics. This was an incredibly ambitious, advanced design

1940, Germany

Konrad Zuse used a similar mechanical approach, but controlled by electrical relays.



PORTION OF BABBAGE'S DIFFERENCE ENGINE.

The Difference Engine, as drawn in Harper's new monthly magazine. Vol. 30, Issue 175, p.34, 1864.

History and Evolution of Computers

Early days of the computer



- Major progress made during World War II (1940's)
- Computer research funded mainly by the War Dept.
- To solve problems related to ballistics or cryptography

History and Evolution of Computers

Early days of the computer

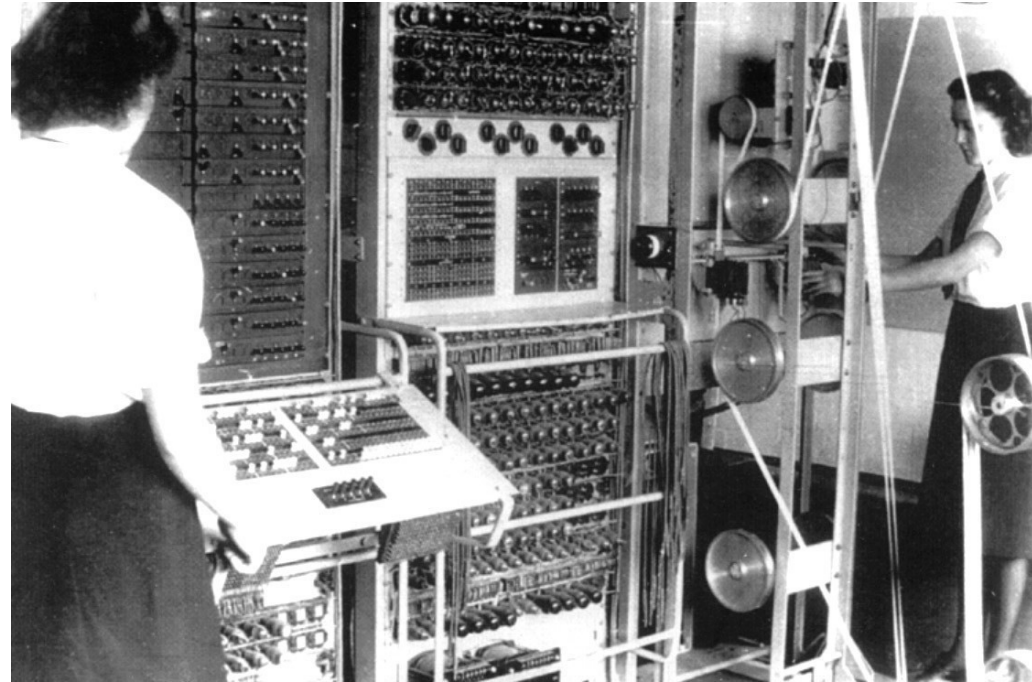
1943, Bletchley Park, UK

Tommy Flowers, working with Alan Turing, built *Colossus* as a code-breaking machine.

There were 10 separate machines built, all were electronic and programmable. However these remained a secret until the year 2000.

1944, USA

ENIAC had a similar design to Colossus, and although it was only operational after Colossus, ENIAC wasn't secret - so was incorrectly hailed for many years as the world's first electronic computer.



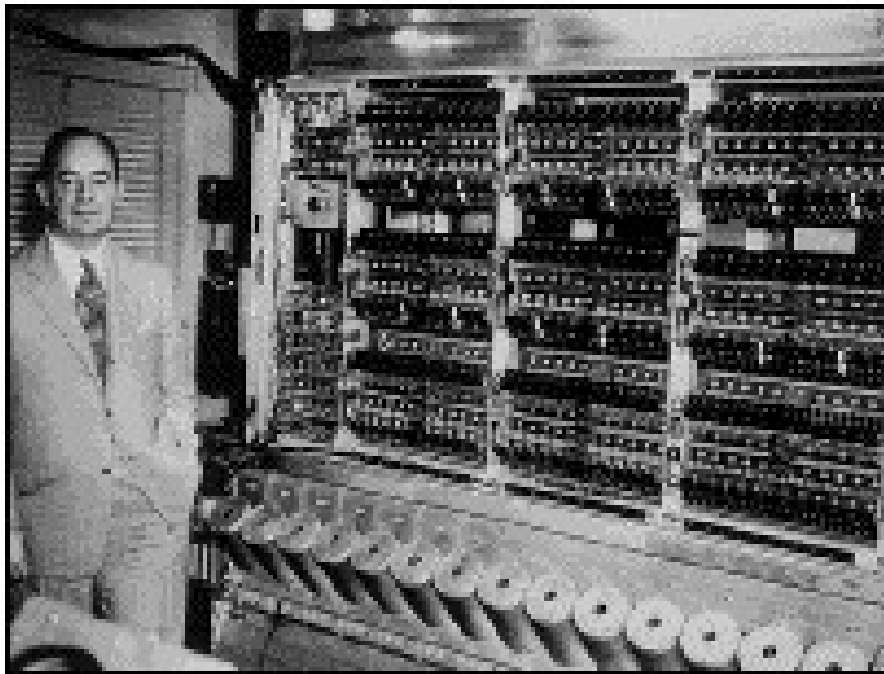
A Colossus computer in use during WWII, courtesy of the Bletchley Park Trust
<http://www.bletchleypark.org.uk>

These were still very much purpose-built computers!!!

History and Evolution of Computers

John Von Neumann

John Von Neumann proposed the **stored program concept** at **Princeton University**.



John von Neumann

History and Evolution of Computers

General Purpose Computers

1948, Manchester

The **SSEM (Baby)** was the first stored program computer (i.e. an architecture like a modern machine).

1953, Manchester

The **transistor computer** was (as the name suggests) the worlds first transistor computer. Manchester was also the home of the world's first commercial computer, in 1951



IBM System/360 photograph, courtesy of Ben Franske, Wikipedia.

By the mid 1950s, the focus of computer development was shifting from the UK to the USA:

1951, USA

MIT (Massachusetts Institute of Technology) built the worlds first real-time computer

1964, USA

IBM enters the picture with the revolutionary System/360.

History and Evolution of Computers

Dawn of the Modern Computer

1971, California, USA

Intel releases the Intel 4004 mass-market single chip CPU. Consider some of its descendants: 8008, 8080, 8080, 8085, 8086, 8088, 80186, 80286, 80386, 80486, Pentium, Pentium II, Pentium III, Pentium IV, Celeron, Core, Atom...

1977, Massachusetts, USA

Digital Equipment Corporation release the VAX11/780; a pioneering computer for one of the greatest American computing companies of all time.



VAX mainframe
<http://williambader.com/museum/vax/vax.html>

By the late 1970s, computers were becoming small and cheap enough that small companies could own one!

Then came home computers...

History and Evolution of Computers

Early “portable” computers



http://en.wikipedia.org/wiki/File:IBM_PC_5150.jpg



Compaq portable



Sinclair ZX Spectrum (photo Bill Bertram)



Apple Lisa

Miniaturisation of the computer

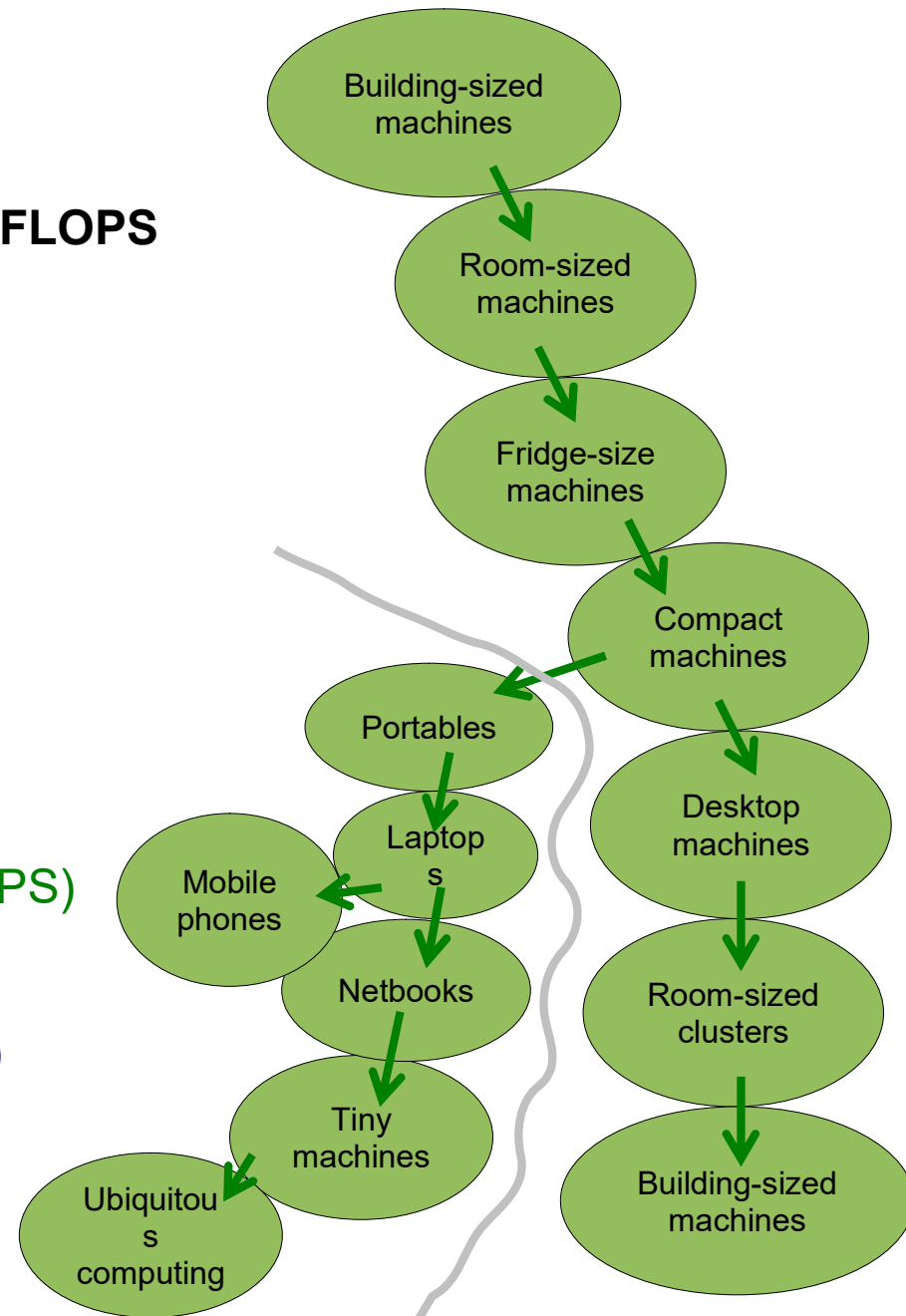
With the advancements in micro-electronics, more transistors can be fabricated into a micro-chip leading to creation of powerful micro-processors and micro-controllers.



History and Evolution of Computers

Evolution in a nut-shell

Year	Floating point operations per second, FLOPS
1941	1
1945	100
1949	1,000 (1 KiloFLOPS, kFLOPS)
1951	10,000
1961	100,000
1964	1,000,000 (1 MegaFLOPS, MFLOPS)
1968	10,000,000
1975	100,000,000
1987	1,000,000,000 (1 GigaFLOPS, GFLOPS)
1992	10,000,000,000
1993	100,000,000,000
1997	1,000,000,000,000 (1 TeraFLOPS, TFLOPS)
2000	10,000,000,000,000
2007	478,000,000,000,000 (478 TFLOPS)
2009	1,100,000,000,000,000 (1.1 PetaFLOPS)



Chapter Contents

- ✓ History and Evolution of Computers
- ✓ **Computer Hardware Decomposition**
- ✓ Classes of Computers
- ✓ Basic operations and requirements
- ✓ Components inside a Computer
- ✓ Example-Register to Register data transfer

Computer Hardware Decomposition

PRE-EXERCISE

There are certain things that a “computer” probably needs, in order to be called a “computer”.

Consider which one of the following are really necessary:

- ☒ a shiny white box with a logo of a half-eaten apple on it
- ☐ a display
- ☐ a hard disc drive
- ☐ DVD/Blu-Ray
- ☐ wireless
- ☐ a unit that processes numbers
- ☐ memory
- ☐ an input device (like a keyboard)
- ☐ a mains power supply or battery

Computer Hardware Decomposition



Do you know what is in the Apple iPad Air?

Computer Hardware Decomposition

iPad Air Teardown

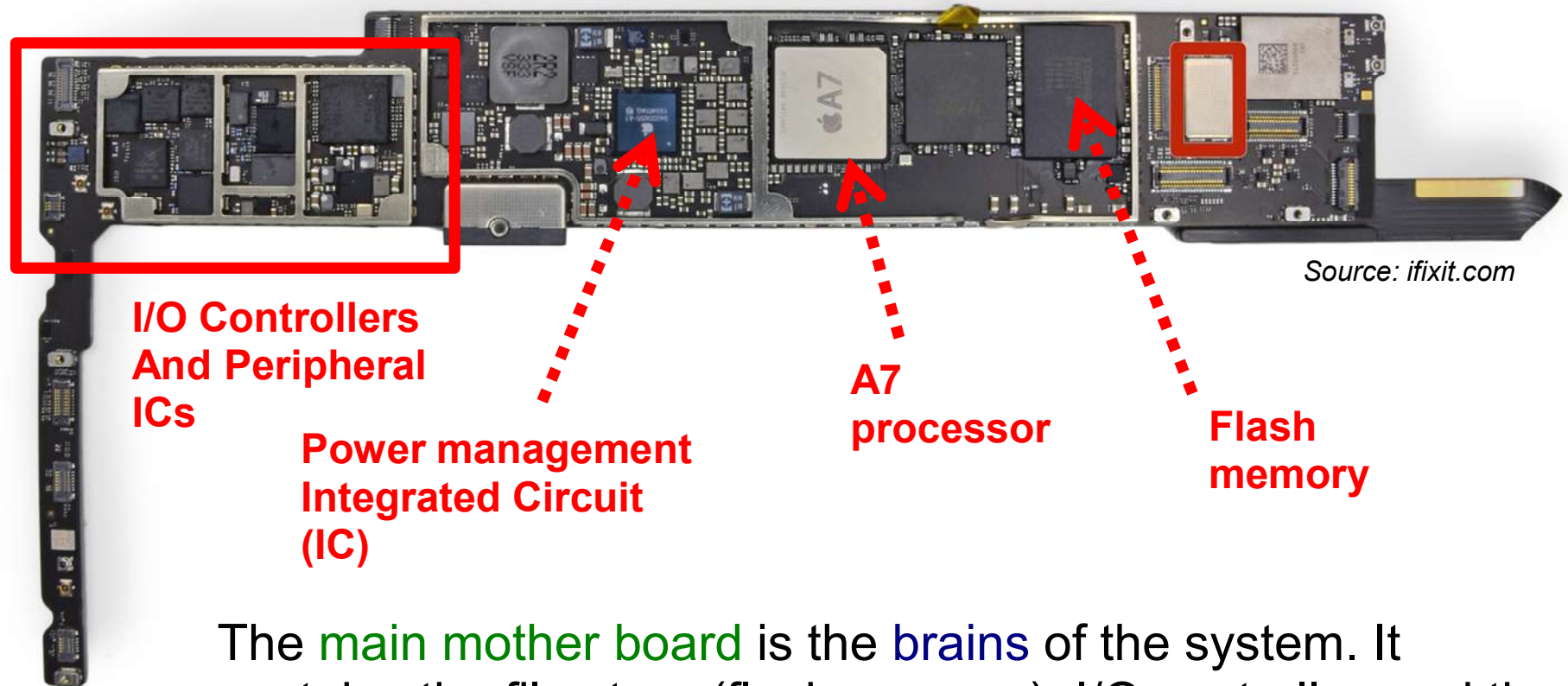


Source: IHS

WHERE IS THE “BRAINS”???

Computer Hardware Decomposition

The brains of the iPad

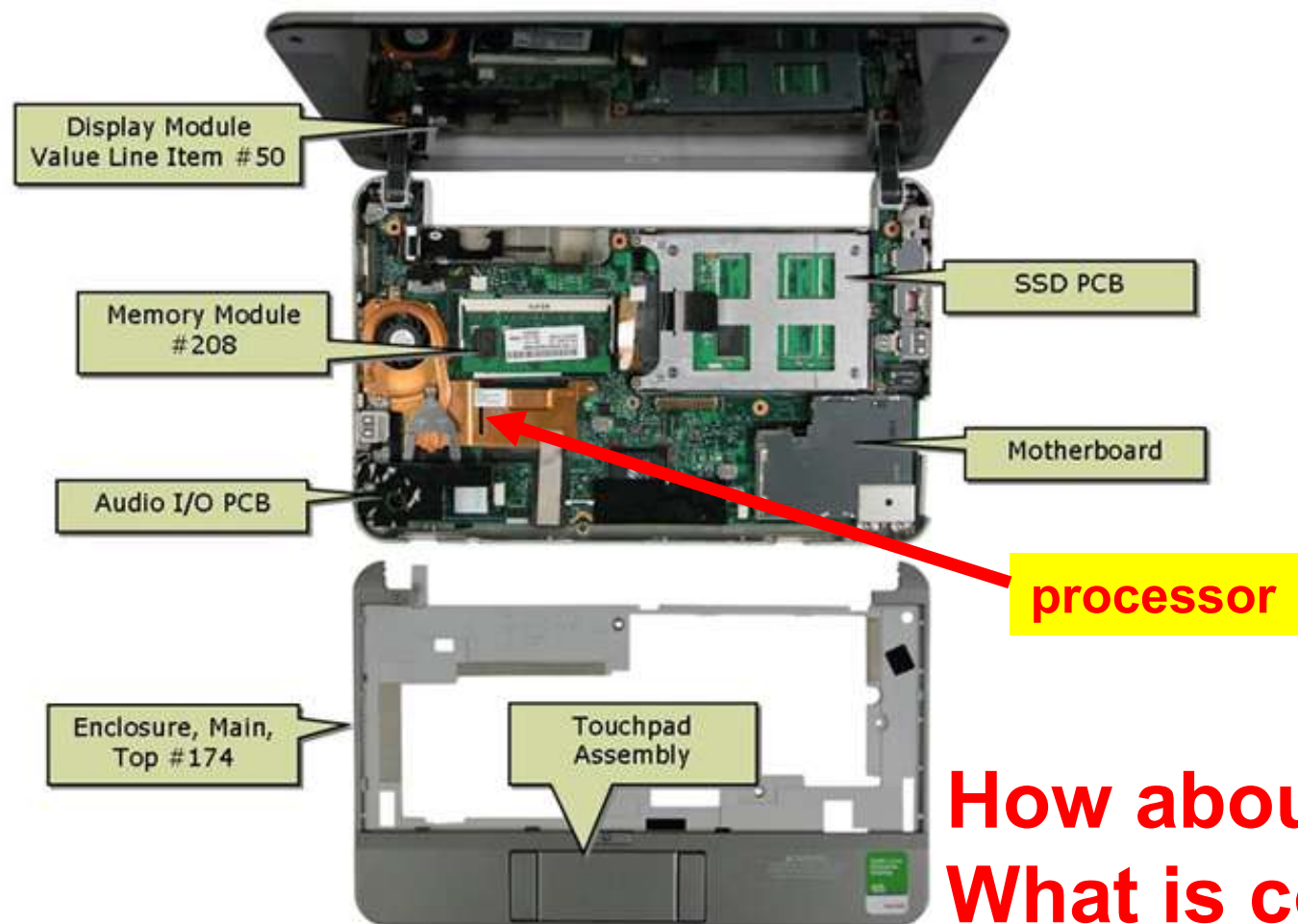


The **main mother board** is the **brains** of the system. It contains the file store (flash memory), I/O controller, and the A7 system-on-chip (SoC) processor.

Computer Hardware Decomposition



Is the iPad hardware decomposition similar to a notebook?



How about a desktop?
What is common?

Computer Hardware Decomposition

Let's Revisit this exercise

There are certain things that a “computer” probably needs, in order to be called a “computer”.

Consider which one of the following are really necessary:

- ☐ a shiny white box with a logo of a half-eaten apple on it
- ☐ a display
- ☐ a hard disc drive
- ☐ DVD/Blu-Ray
- ☐ wireless
- ☐ a unit that processes numbers
- ☐ memory
- ☐ an input device (like a keyboard)
- ☐ a mains power supply or battery

Chapter Contents

- ✓ History and Evolution of Computers
- ✓ Computer Hardware Decomposition
- ✓ **Classes of Computers**
- ✓ Basic operations and requirements
- ✓ Components inside a Computer
- ✓ Example-Register to Register data transfer

Classes of Computers

- From supercomputer → personal computer
→ personal digital assistant → digital watch.
- All these devices contain some form of computational element(s).
- There is no clear cut way to classify all these computers. But we review three broad categories:
 - Supercomputers, microcomputers (multi-chip) and embedded computer (single-chip).



Supercomputer



Microcomputer



Embedded

Embedded microcomputers

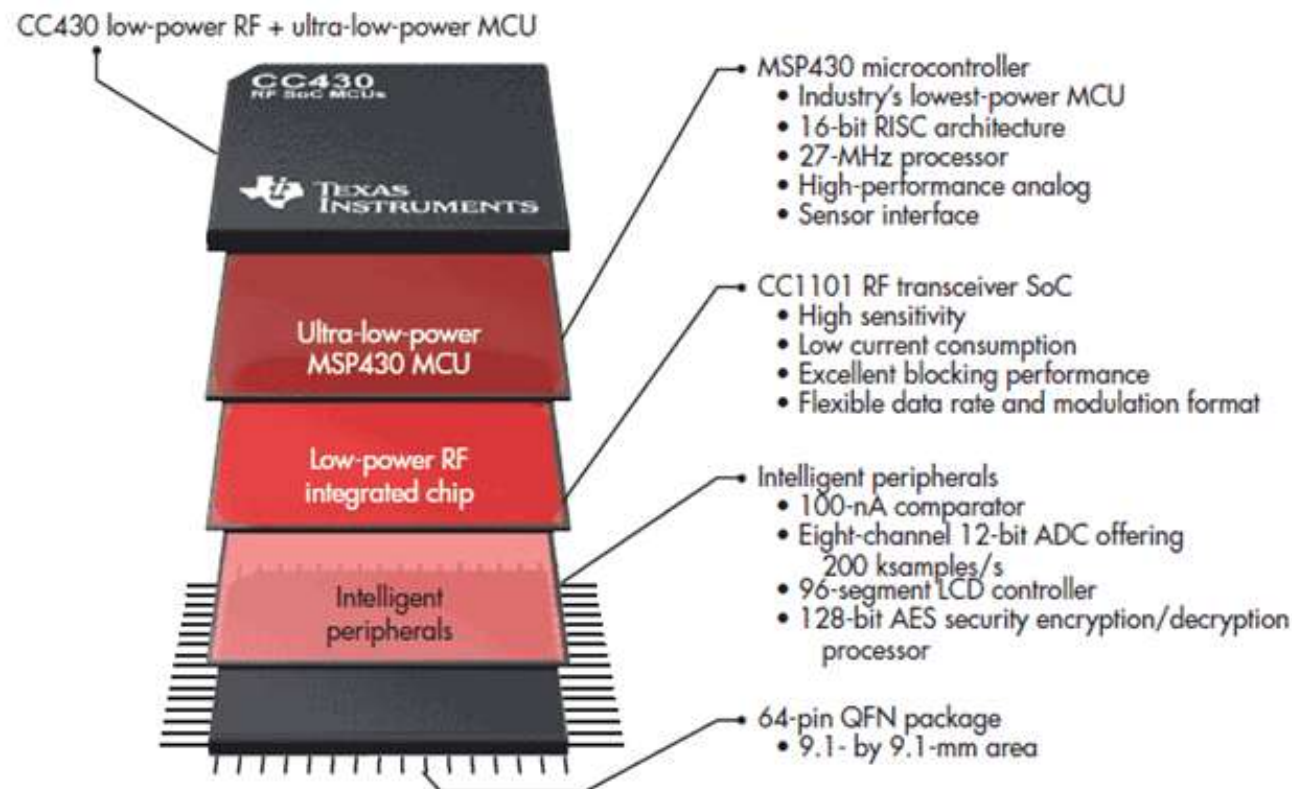
- Compact computers that use a single-chip (**microcontroller**) that contains the processing unit, memory and required peripheral support.
- They are called **embedded** microcomputers as the presence of the microprocessor is non-obvious. Embedded systems are all around us.



Examples of embedded microcomputer applications

Systems-on-Chip (SoC)

A microcontroller that has been integrated with other interface systems on a single chip is known as Systems-on-Chip (SoC).



The highly integrated Texas Instruments CC430 contains an MCU, a transceiver, and intelligent peripherals, all within a 9.1- by 9.1-mm, 64-pin, QFN package. These parameters suit it for the design of home portable healthcare equipment.

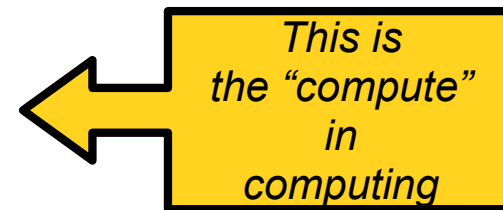
Chapter Contents

- ✓ History and Evolution of Computers
- ✓ Computer Hardware Decomposition
- ✓ Classes of Computers
- ✓ **Basic operations and requirements**
- ✓ Components inside a Computer
- ✓ Example-Register to Register data transfer

Basic Operations and Requirements

Actually, at it's heart, a computer is simply a device that does 3 things repetitively:

1. Stores and retrieves (loads) values (data)
2. Transforms those values
3. Transfers those values from one place to another



Every computer program ever written is just a sequence of these three operations.

Basic Operations and Requirements

The things that differ between computers are really:

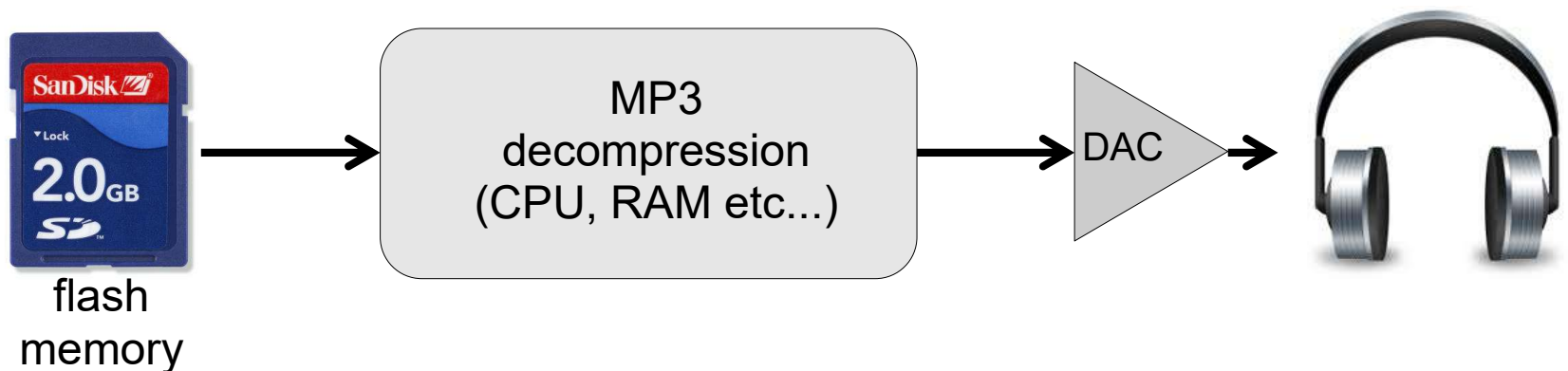
1. Where the values can be stored
(and whether different numbers convey different meanings)
2. What type of processing can be done
3. Where can the data be transferred from or to

Of course there are some other things to consider like: Speed, accuracy, data size, cost, power consumption, physical size, programmability, ease of debugging, memory size, input output capabilities.

Basic Operations and Requirements

Example 1

An MP3 player during audio playback - loads data from flash memory into RAM in MP3 format. The CPU decompresses the MP3 data into audio samples. The audio samples are transferred to a digital-to-analogue converter output.

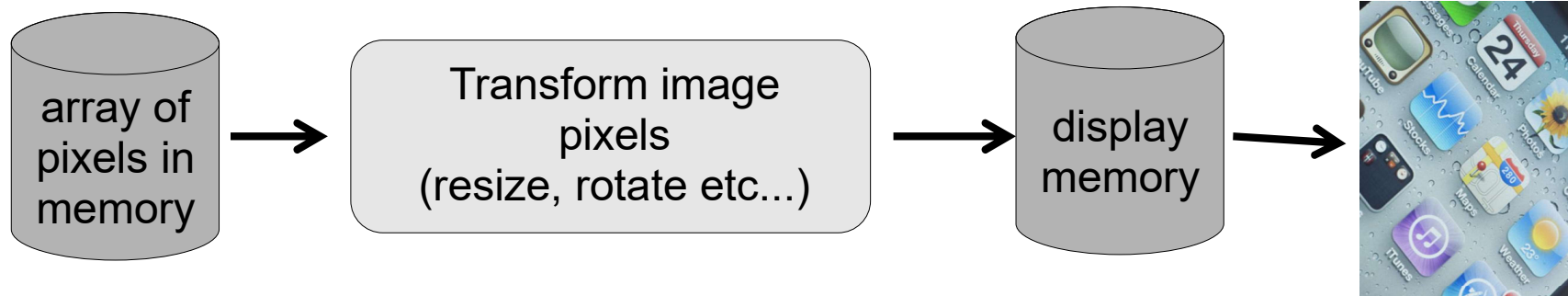


LOAD→PROCESS→OUTPUT

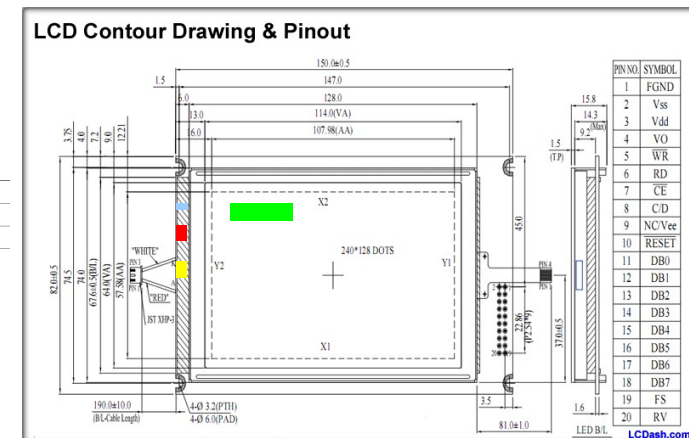
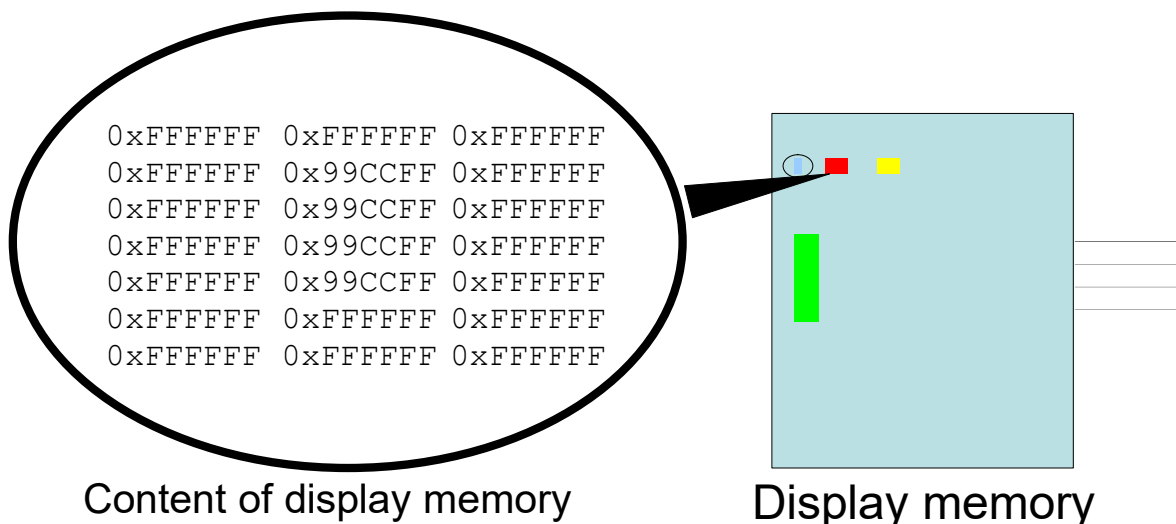
Basic Operations and Requirements

Example 2

A games console, displaying an icon on an LCD screen...



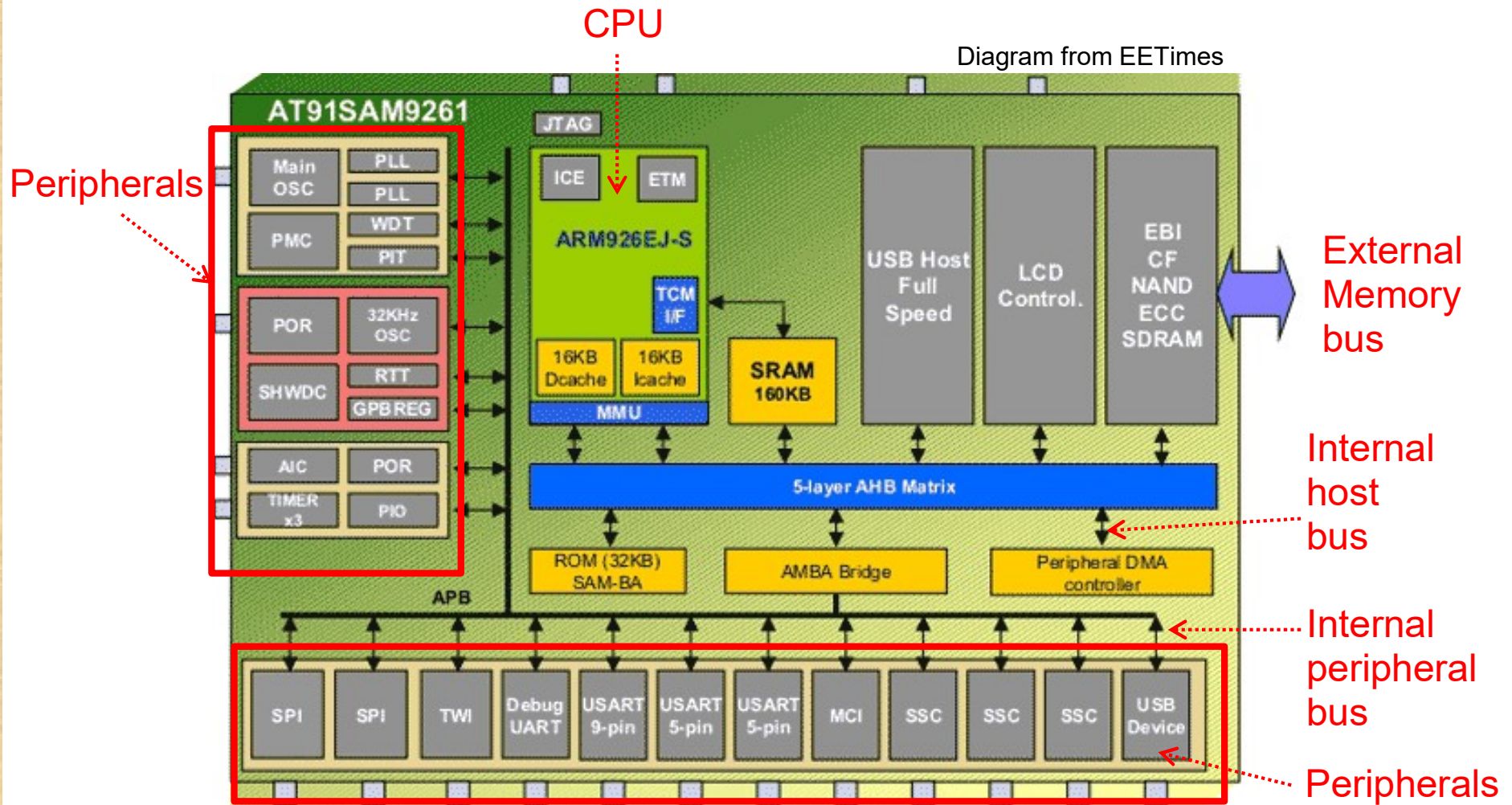
LOAD→PROCESS→OUTPUT



Chapter Contents

- ✓ History and Evolution of Computers
- ✓ Computer Hardware Decomposition
- ✓ Classes of Computers
- ✓ Basic operations and requirements
- ✓ **Components inside a Computer**
- ✓ Example-Register to Register data transfer

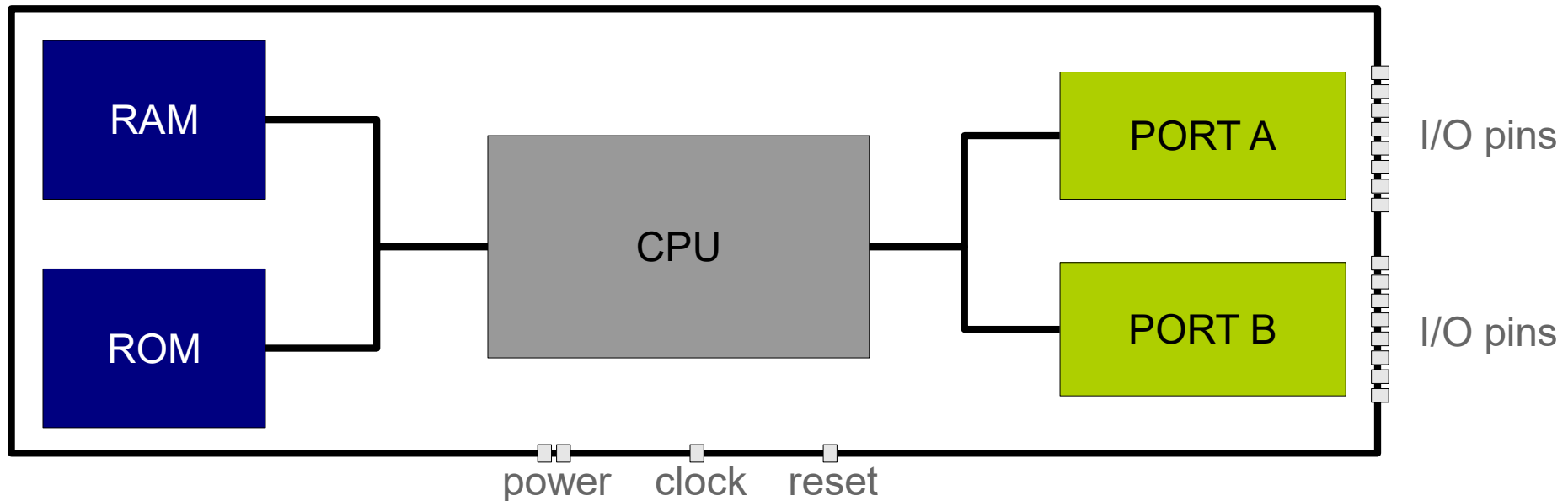
Components inside a Computer



Here is another system-on-chip device... All this is inside one IC chip We are starting to see clustering of devices inside the chip.

Components inside a Computer

Using a Micro-controller as reference



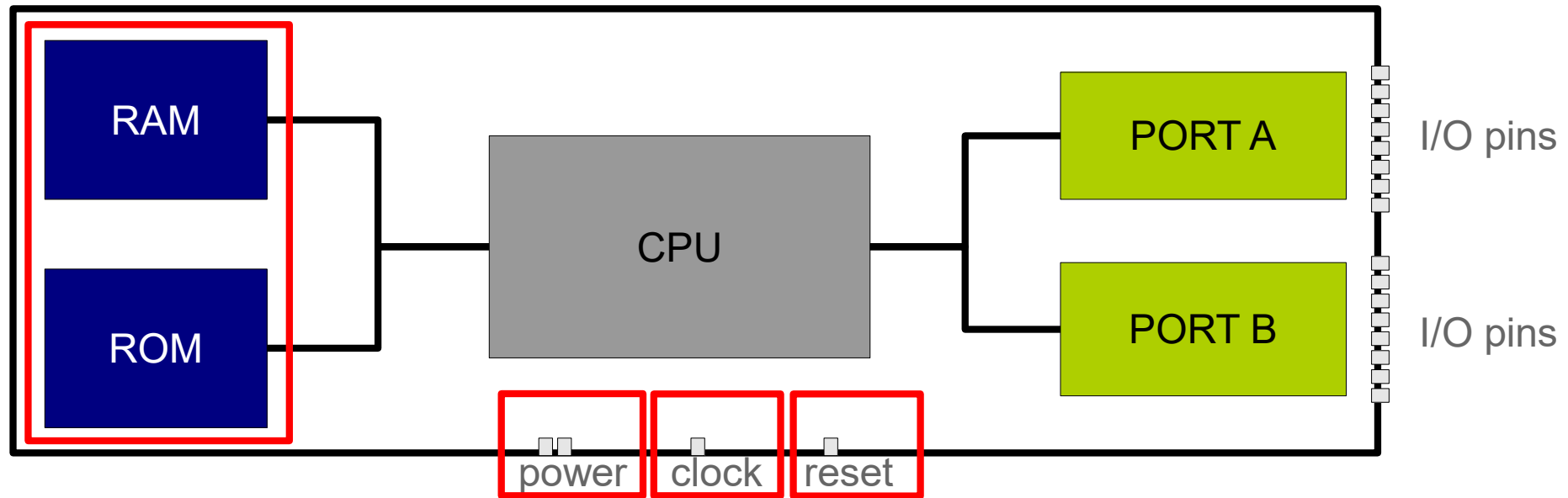
By contrast, this is a diagram of a simple PIC microcontroller IC. Notice how few things are inside compared to the ARM SoC in the previous slide!

However, this still supports the basic functionality of a computer:

- Read only memory (ROM) to hold a **program**.
- Random access memory (RAM) to hold **variables** and **data**.
- An arithmetic logic unit (ALU) to **process** them.
- Two ports for **input/output**

Components inside a Computer

Using a Micro-controller as reference



Apart from the CPU itself, the most important considerations are probably **Power**, **Clock**, **Reset** and **Memory**...

Just an overview here as more details will be discussed in the following chapters!!!

Components inside a Computer

The Clock

Almost all modern computers are **synchronous** - which means that all their internal signals are **triggered on clock edges**... it is a very fundamental technique in making digital logic **reliable**. This is also one of the challenges making faster processors.

Complex modern CPUs have **one master clock**, but use **dividers** and **multipliers** to derive several different speed clocks from this.

Generally, things closer to the core of the CPU (registers and ALU) will be clocked faster. Things further away will clock slower.

External memory will be a bit slower again, and most peripherals will be the slowest.

For similar processors, the higher the clock speed, the faster the processing. Is this true when comparing different processors?

Components inside a Computer

Reset

Logic bits are stored as **voltage** in **flip-flops**, **latches** and as **electronic charge** in a **capacitive cell**.

When you start up a logic device (by turning on the power), you might guess that all the voltages and charges start at zero. But it's often not true!

So a **reset signal** is very important to give a CPU a clean start.

One of the things that will get reset to zero when most CPUs start up is the program counter (PC).

It causes a CPU to boot from address zero: we will see this later.

This is a hardware reset. How about a software reset?

Components inside a Computer

Memory

Non-volatile memory

that retains its contents when the power is turned off. This can be programmable (like flash, EEPROM, EPROM), or fixed (like ROM and mask-programmed ROM).

Volatile memory

that gets wiped whenever the power goes off. This includes SRAM and DRAM (including SDRAM, DDR etc...).

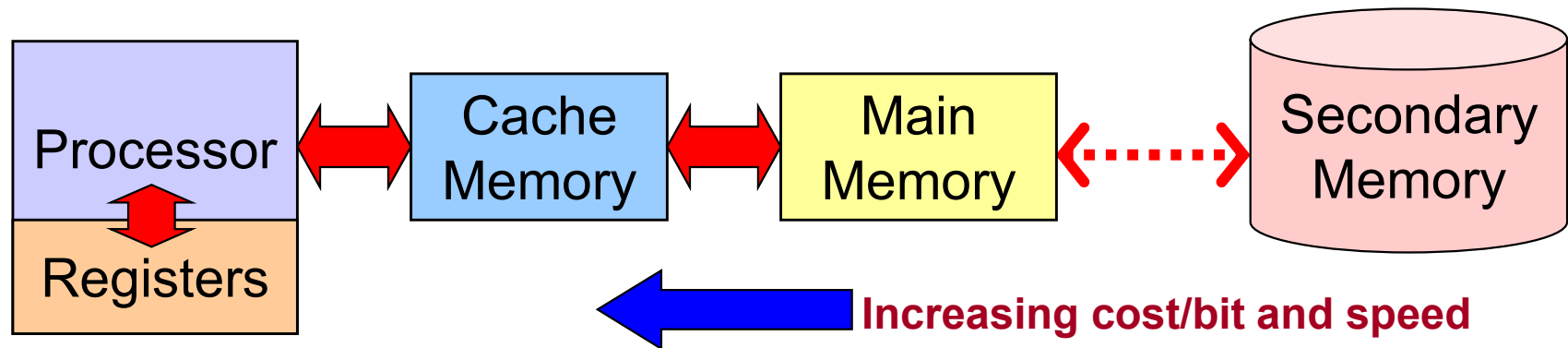
Note: technically, hard disc, CD-ROM and DVD-ROM are non-volatile memory (either read-only or read-write).

What about write-only memory?

Components inside a Computer

Memory

Memories are generally organized in levels of decreasing speed and cost/bit.



- Registers** Very fast access but limited numbers within CPU. Operates at CPU clock rate (size: 2-128 registers)
- Cache Memory** Fast access static RAM close to CPU. Typical access time 8-35nS (size: 1kB - 512 kB)
- Main memory** Usually dynamic RAM or ROM (for program storage). Typical access time 20-100nS. (size: 1kB – 1GB)
- Secondary Memory** Not always random access but non-volatile. Maybe be based on magnetic or flash technology. Typical access time 5-20mS. (size: 1MB - 80GB)

Chapter Contents

- ✓ History and Evolution of Computers
- ✓ Computer Hardware Decomposition
- ✓ Classes of Computers
- ✓ Basic operations and requirements
- ✓ Components inside a Computer
- ✓ **Example-Register to Register data transfer**

Example

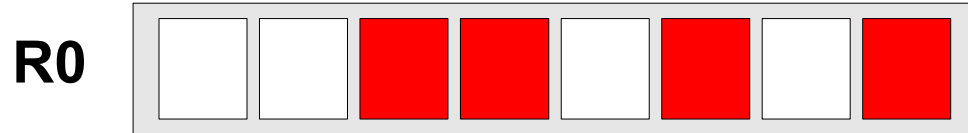
Register-to-Register Data Transfer

To highlight the use of the clock and show an example of memory load/store using register-to-register data transfer.

We start by defining a register that can hold 8 bits of data:



We'll call this register R0, and then put some data inside it:



Positive Voltage = Logic 1, No voltage = Logic 0

If red represents a positive voltage, what is the value stored in the register???

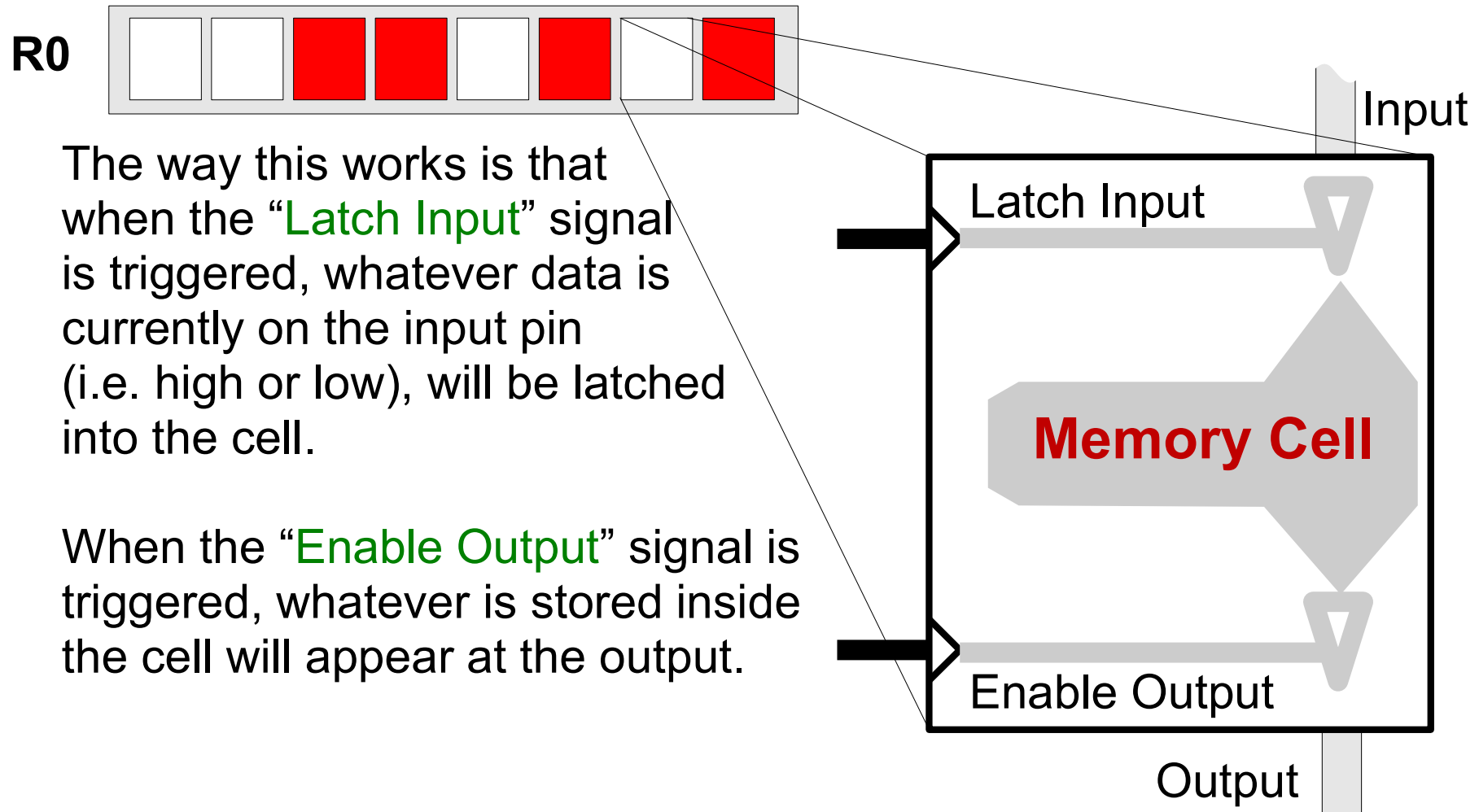
It contains the binary value 00110101

We need to know a little bit about how the register operates now...

Example

Register-to-Register Data Transfer

The register actually has some **control wires**, and some **data connections**. Let's look at a single bit from the register.

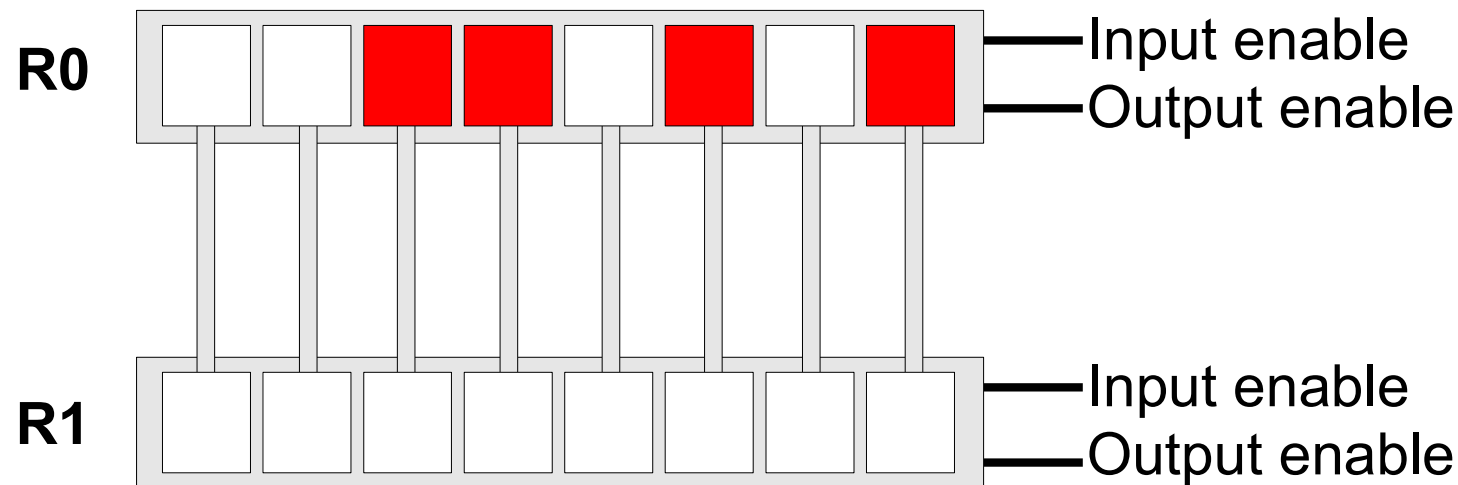


Example

Register-to-Register Data Transfer

So let's see how we can transfer data from R0 to a new register, we will call R1.

First let's run wires from each cell in R0 to the corresponding cell in R1:

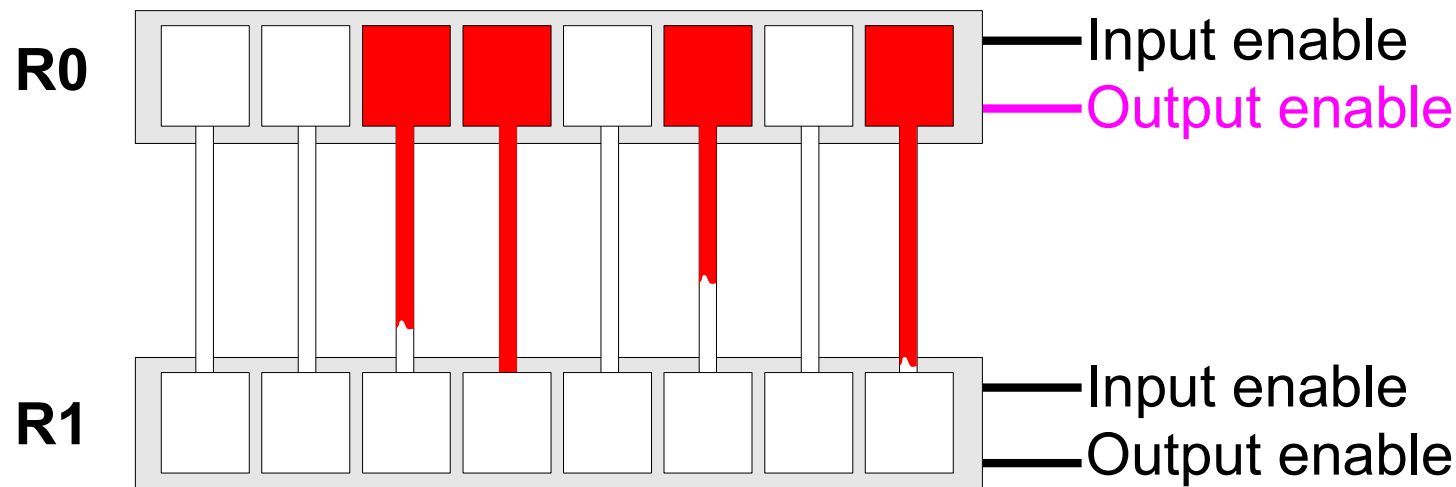


Example

Register-to-Register Data Transfer

Next is a sequence of operations.

1. We enable the output of R0:



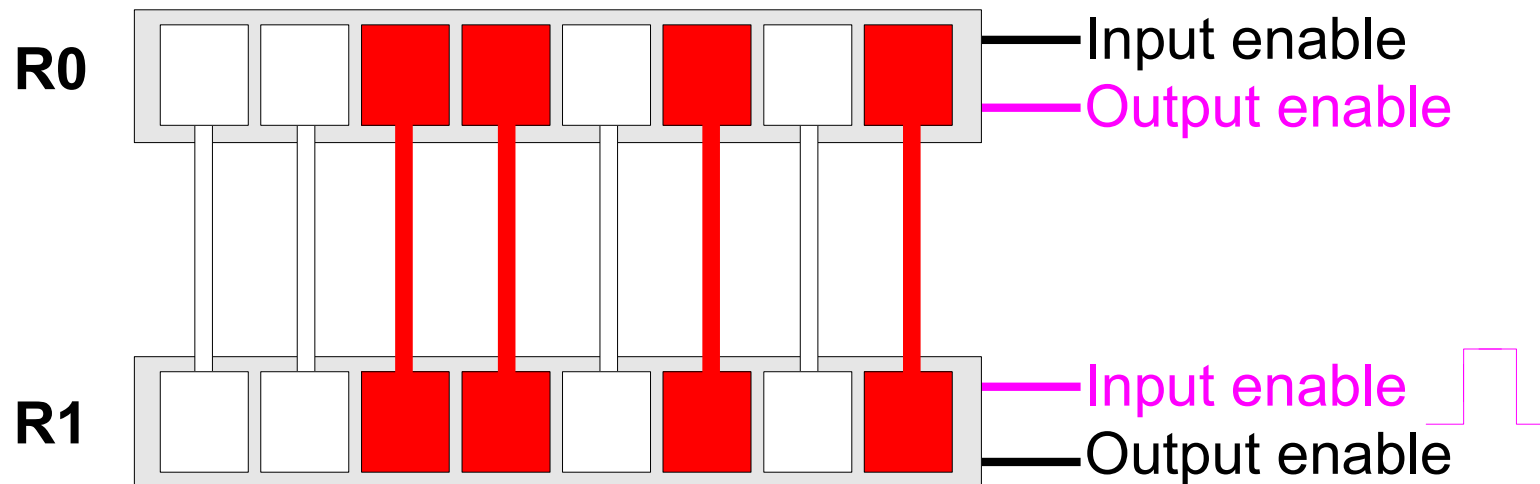
This causes whatever voltage is held in each bit position of R0 to flow through the wires connected to the R0 output pins.

But it can't enter R1 yet...

Example

Register-to-Register Data Transfer

2. Once the voltage on those wires has stabilised, we momentarily trigger the R1 input enable (keeping R0 output turned on):

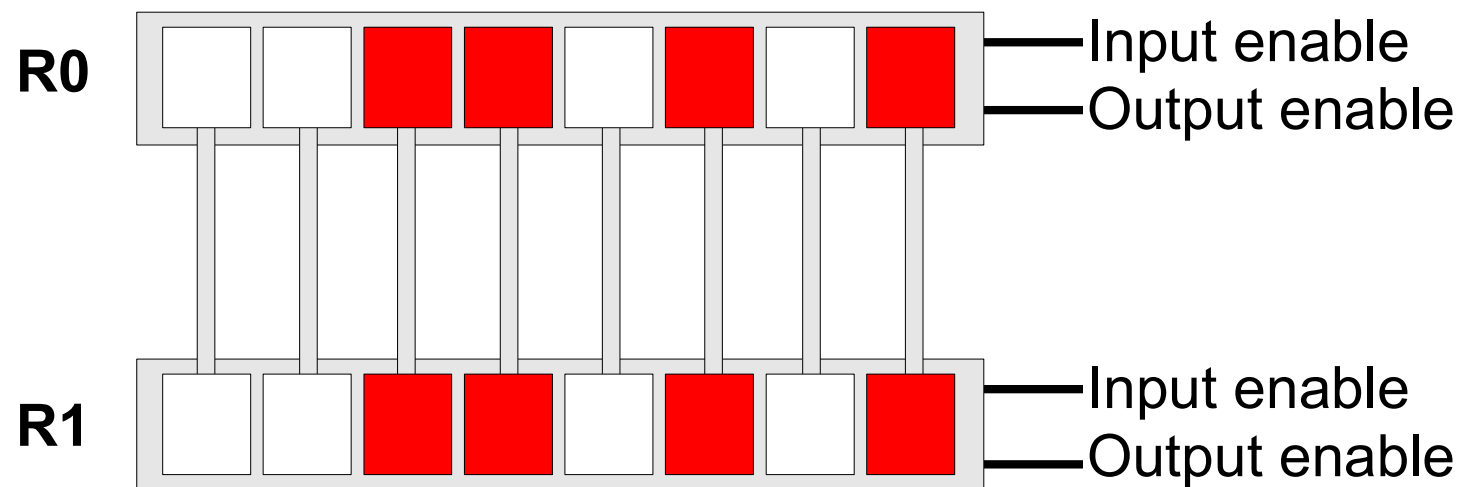


So now, the data gets into R1, and is latched there.

Example

Register-to-Register Data Transfer

3. Now we can safely turn off the R0 output enable:



Example

Register-to-Register Data Transfer

Before we explore further. Just a reminder:

All data is just 1's and 0's. It doesn't matter if the data is a single bit, a byte, word, character, string, program, or your entire set of MP3s and BluRay movies... they are just 1's and 0's to a computer.

What matters is:

Where is the data stored?

How is the data stored?

Where does it need to be transferred to?

How does it need to be processed?

How much data is there?

How quickly does it need to be processed?

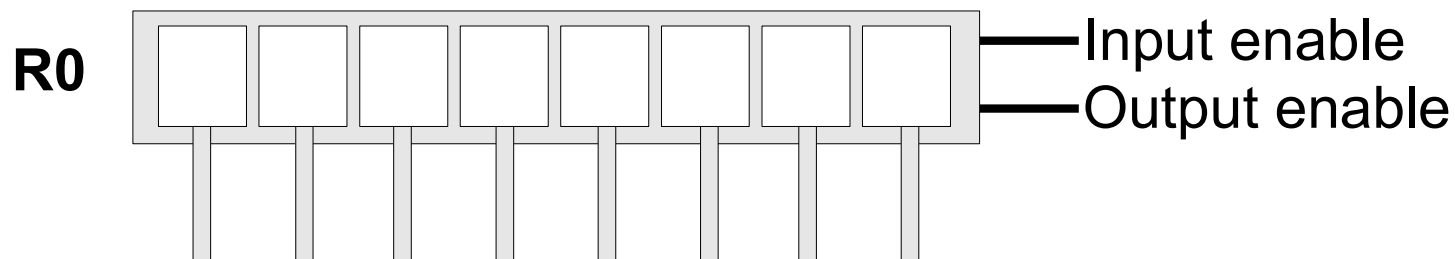
How important is the data and it's timeliness?

How the programmer/user interprets the data?

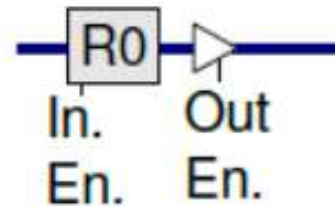
Example

Register-to-Register Data Transfer

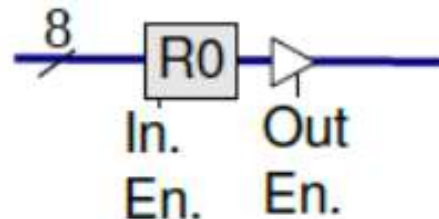
Going back to our example. I don't want to have to draw so much every time I discuss register operation... So we **simplify**:



It's a lot easier to draw it in this way (note the bus is now a single thick line):



Normally we would indicate the data bus “width” (i.e. 8 bits in this case), as follows:

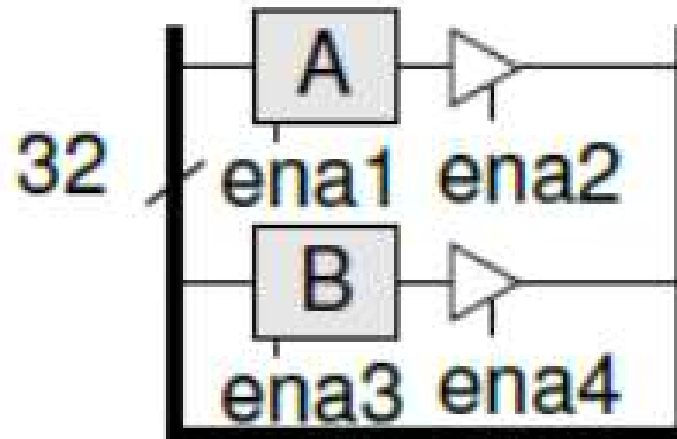


Example

Register-to-Register Data Transfer

Such **buses** are used extensively to transfer data to and from registers, memory, input/output ports and so on.

Tristate buffers (▷) arbitrate between these (we had some of these inside our register*), all timed by a CPU clock, and all controlled by a **CPU control unit**.



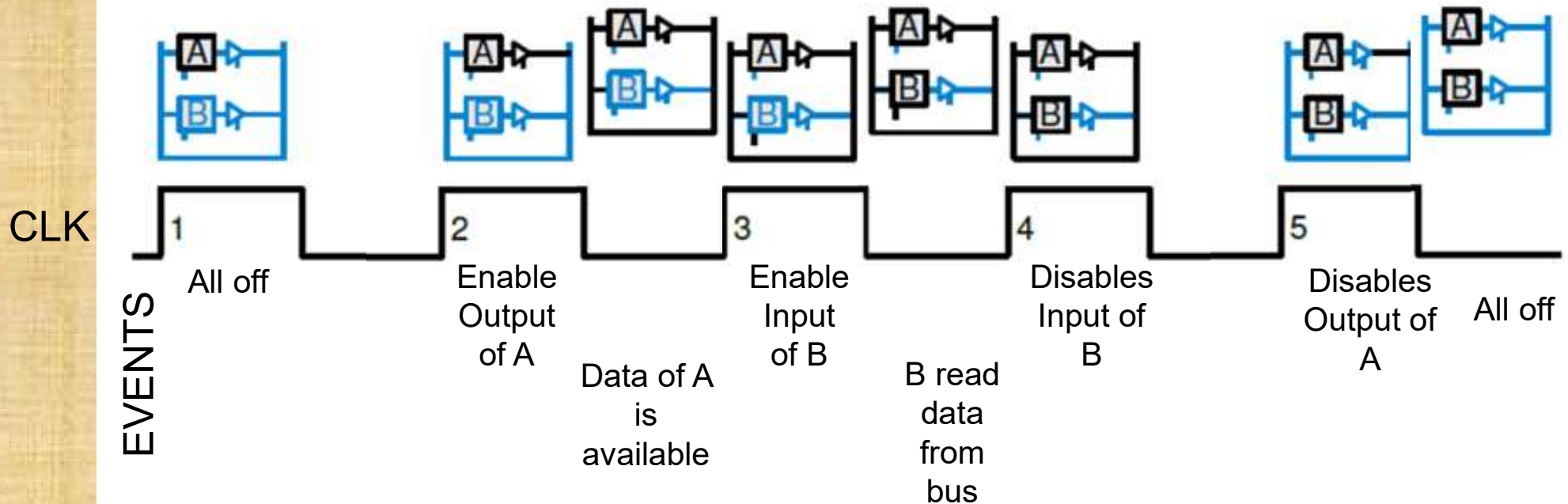
**Tristate buffers mean the output wire can be HIGH, LOW or OFF. They are very important in allowing different units to share the same bus to convey data.*

Example

Register-to-Register Data Transfer

Here is an example showing how data in one register (A) can be transferred over a shared bus to another register (B) using the clock.

Darker lines indicates that the particular bus or signal is active at that time.



In this example, 5 separate clock cycles are used to **enable** and **disable** buses, **tristates** and **latch inputs** at the rising clock edges **in the correct sequence** to **transfer data** from register A to register B.

Summary

LEARNING OUTCOMES OF CHAPTER:

- ✓ Knowledge about the evolution of computing
- ✓ Ability to identify essential components of a computer
- ✓ Ability to determine the class of a computer
- ✓ Understand the basic operations of a computer

NEXT UP: Details of the CPU